# WebApi Handson Solution

# 6. WebApi Hands-on Solution

## Objectives Addressed

* Introduction to Kafka (Architecture, Topics, Partitions, Brokers).
* Kafka plugin in .NET.
* Hands-on: Create a Chat Application using C# with Kafka as a streaming platform.

## Solution Implementation

### 1. Prerequisites & Kafka Setup

Before writing the code, ensure Kafka and Zookeeper are running.

**Start Zookeeper:**
```cmd
zookeeper-server-start.bat ../../config/zookeeper.properties
```

**Start Kafka Server:**
```cmd
kafka-server-start.bat ../../config/server.properties
```

**Create a Topic:**
```cmd
kafka-topics.bat --create --topic chat-topic --bootstrap-server localhost:9092
```

### 2. C# .NET Kafka Implementation

First, install the Nuget package:
```bash
dotnet add package Confluent.Kafka
```

# WebApi Handson Solution

#### Producer (Publishing Chat Messages)

```csharp
using System;
using Confluent.Kafka;
using System.Threading.Tasks;

class Producer
{
    public static async Task Run()
    {
        var config = new ProducerConfig { BootstrapServers = "localhost:9092" };

        using (var producer = new ProducerBuilder<Null, string>(config).Build())
        {
            while (true)
            {
                Console.Write("Enter message: ");
                string text = Console.ReadLine();
                if (text == "exit") break;

                var result = await producer.ProduceAsync("chat-topic", new Message<Null, string> { Value = text });
                Console.WriteLine($"Delivered '{result.Value}' to '{result.TopicPartitionOffset}'");
            }
        }
    }
}
```

#### Consumer (Reading Chat Messages)

```csharp
using System;
using Confluent.Kafka;
```

# WebApi Handson Solution

```
using System.Threading;

class Consumer
{
    public static void Run()
    {
        var config = new ConsumerConfig
        {
            BootstrapServers = "localhost:9092",
            GroupId = "chat-group",
            AutoOffsetReset = AutoOffsetReset.Earliest
        };

        using (var consumer = new ConsumerBuilder<Ignore, string>(config).Build())
        {
            consumer.Subscribe("chat-topic");
            CancellationTokenSource cts = new CancellationTokenSource();

            Console.WriteLine("Consuming messages... (Ctrl+C to quit)");
            try
            {
                while (true)
                {
                    var cr = consumer.Consume(cts.Token);
                    Console.WriteLine($"Received message: {cr.Message.Value}");
                }
            }
            catch (OperationCanceledException)
            {
                consumer.Close();
            }
        }
    }
}
```

# WebApi Handson Solution

## Testing the Chat Application

1. Start the Producer application in one command prompt.
2. Start the Consumer application in another command prompt.
3. Type messages in the Producer prompt; you should instantly see them printed in the Consumer prompt, demonstrating a successful Kafka stream setup.